

Machine Learning

MSE FTP MachLe

Christoph Würsch



Lab 4, A3: Yelp Business Classification using Bag-of-Words and tf-idf

Yelp is a crowd-sourced review forum, as well as an American multinational corporation headquartered in San Francisco, California. It develops, hosts and markets Yelp.com and the Yelp mobile app, which publish crowd-sourced reviews about local businesses, as well as the online reservation service Yelp Reservations. The company also trains small businesses in how to respond to reviews, hosts social events for reviewers, and provides data about businesses, including health inspection scores. The data is open and can be downloaded here <https://www.yelp.com/dataset/challenge>.

We have prepared a partial dataset from the big Yelp dataset in the **pickle serialization format** that you can read into a pandas dataframe using `pd.read_pickle`.

```
In [18]: import json
import numpy as np
import pandas as pd
from sklearn.feature_extraction import text
from sklearn.linear_model import LogisticRegression
import sklearn.model_selection as model
import sklearn.preprocessing as preproc
```

(a) Load and prep Yelp reviews data

```
In [19]: DataPath='D:/Downloads/YelpDataset/'
nightlife_subset = pd.read_pickle('nightlife_subset.pkl')
restaurant_subset = pd.read_pickle('restaurant_subset.pkl')
```

(b) combine both datasets

```
In [20]: combined = pd.concat([nightlife_subset, restaurant_subset])

In [21]: combined['target'] = combined.apply(lambda x: 'Nightlife' if x['categories'] == 'nightlife',
axis=1)

In [22]: combined
```

Out[22]:

	business_id	name	stars_y	text	categories	target
2203299	lpYFsXFrojiBZ1kbWR2lZw	Four Peaks Grill & Tap	5	Great service and food... Enjoy the atmosphere...	Food, Restaurants, American (New), Local Flavo...	True
482774	KskYqH1Bi7Z_61pH6Om8pg	Lotus of Siam	5	Lotus is one of my all time favorites in Las V...	Wine Bars, Nightlife, Restaurants, Seafood, Ca...	True
3086879	zFnPRtP7LGvr3sfxy_dfg	Revolution Ale House	1	This place has definitely gone downhill. We fi...	Nightlife, Italian, Restaurants, Pizza, Bars	True
512115	gUR2pWQKLpgMEm_R_al_aw	Shooters On The Water	3	Thought this might be a worn out hang out for ...	Dive Bars, American (New), American (Tradition...	True
3278073	hmYnzs8-aHbltaOOGDgmbA	Zipps Sports Grill	4	Zipps is almost always on happy hour! They hav...	Sports Bars, American (Traditional), Nightlife...	True
...	...	...	...	...	...	...
2429937	Vs7gc9EE3k9wARuUcN9piA	Pan Asian	5	Pan Asian stays 5 stars for me. I went back t...	Thai, Chinese, Japanese, Restaurants	False
640067	mYzLPKXvOVRrQivHnDqD5g	YamChops	5	Amazingly delicious with a variety of distinct...	Butcher, Juice Bars & Smoothies, Vegan, Restau...	False
1566257	9ZI4uWSgSMpxHnsK_MPneg	Palermo Family Restaurant	4	Eat here quite often, the Palermo Special piz...	Restaurants, Pizza	False
1607254	F-AYOq1xIY2u_qmWUG5VBw	Hakka Ren	5	It was amazing! I had the chilli fish schezuan...	Chinese, Halal, Indian, Restaurants	False
1076163	yNBVOKSZN_AljSjdHf_rqA	Garbanzo Mediterranean Grill	4	My salad was really good. Nice atmosphere, fri...	Falafel, Event Planning & Services, American (...)	False

20000 rows × 6 columns

In [23]: `print(combined.shape)`

(20000, 6)

(c) Split the dataset into a training and test set

```
In [24]: # Split into training and test data sets
training_data, test_data = model.train_test_split(combined,
                                                    train_size=0.7,
                                                    random_state=123)
```

In [25]: `training_data.shape`

Out[25]: (14000, 6)

In [26]: `test_data.shape`

Out[26]: (6000, 6)

(d) Transform the text as BoW (bag-of-words)

```
In [27]: # Represent the review text as a bag-of-words
bow_transform = text.CountVectorizer()
X_tr_bow = bow_transform.fit_transform(training_data['text'])
```

In [28]: `len(bow_transform.vocabulary_)`

Out[28]: 29944

In [29]: `X_tr_bow.shape`

Out[29]: (14000, 29944)

In [30]: `X_te_bow = bow_transform.transform(test_data['text'])`

```
In [31]: y_tr = training_data['target']
y_te = test_data['target']
```

(e,f) Classify with logistic regression

```
In [32]: def simple_logistic_classify(X_tr, y_tr, X_test, y_test, description, _C=1.0):
    ## Helper function to train a logistic classifier and score on test data
    m = LogisticRegression(C=_C, solver='newton-cg', max_iter=500).fit(X_tr, y_tr)
    s = m.score(X_test, y_test)
```

```
print ('Test score with', description, 'features:', s)
return m
```

```
In [33]: m1 = simple_logistic_classify(X_tr_bow, y_tr, X_te_bow, y_te, 'bow')
```

Test score with bow features: 0.7145

(f) applying normalization to the features

```
In [34]: X_tr_l2 = preproc.normalize(X_tr_bow, axis=0)
X_te_l2 = preproc.normalize(X_te_bow, axis=0)
```

```
In [35]: m2 = simple_logistic_classify(X_tr_l2, y_tr, X_te_l2, y_te, 'l2-normalized')
```

Test score with l2-normalized features: 0.739

(g) tf-idf representation

```
In [36]: # Create the tf-idf representation using the bag-of-words matrix
tfidf_trfm = text.TfidfTransformer(norm=None)
X_tr_tfidf = tfidf_trfm.fit_transform(X_tr_bow)
```

```
In [37]: X_te_tfidf = tfidf_trfm.transform(X_te_bow)
```

```
In [38]: m3 = simple_logistic_classify(X_tr_tfidf, y_tr, X_te_tfidf, y_te, 'tf-idf')
```

Test score with tf-idf features: 0.6798333333333333

(h) Tune regularization parameters using grid search

```
In [42]: param_grid_ = {'C': [1e-5, 1e-3, 1e-1, 1e0, 1e1, 1e2]}
bow_search = modsel.GridSearchCV(LogisticRegression(solver='lbfgs',max_iter=1000),
                                cv=5, param_grid=param_grid_, return_train_score=True)
l2_search = modsel.GridSearchCV(LogisticRegression(solver='lbfgs',max_iter=1000),
                                cv=5, return_train_score=True, param_grid=param_grid_)
tfidf_search = modsel.GridSearchCV(LogisticRegression(solver='lbfgs',max_iter=1000),
                                   cv=5, return_train_score=True, param_grid=param_grid_)
```

```
In [25]: bow_search.fit(X_tr_bow, y_tr)
```

```
C:\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:940: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
C:\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:940: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
C:\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:940: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
C:\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:940: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
C:\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:940: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
C:\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:940: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
C:\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:940: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
C:\Anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:940: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
 extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)

```
Out[25]: GridSearchCV(cv=5, error_score=nan,
                    estimator=LogisticRegression(C=1.0, class_weight=None, dual=False,
                                                fit_intercept=True,
                                                intercept_scaling=1, l1_ratio=None,
                                                max_iter=1000, multi_class='auto',
                                                n_jobs=None, penalty='l2',
                                                random_state=None, solver='lbfgs',
                                                tol=0.0001, verbose=0,
                                                warm_start=False),
                    iid='deprecated', n_jobs=None,
                    param_grid={'C': [1e-05, 0.001, 0.1, 1.0, 10.0, 100.0]},
                    pre_dispatch='2*n_jobs', refit=True, return_train_score=True,
                    scoring=None, verbose=0)
```

```
In [43]: bow_search.best_score_
```

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_27864\810077298.py in <module>
----> 1 bow_search.best_score_

AttributeError: 'GridSearchCV' object has no attribute 'best_score_'
```

```
In [27]: l2_search.fit(X_tr_l2, y_tr)
```

```
Out[27]: GridSearchCV(cv=5, error_score=nan,
                    estimator=LogisticRegression(C=1.0, class_weight=None, dual=False,
                                                fit_intercept=True,
                                                intercept_scaling=1, l1_ratio=None,
                                                max_iter=500, multi_class='auto',
                                                n_jobs=None, penalty='l2',
                                                random_state=None, solver='lbfgs',
                                                tol=0.0001, verbose=0,
                                                warm_start=False),
                    iid='deprecated', n_jobs=None,
                    param_grid={'C': [1e-05, 0.001, 0.1, 1.0, 10.0, 100.0]},
                    pre_dispatch='2*n_jobs', refit=True, return_train_score=True,
                    scoring=None, verbose=0)
```

```
In [49]: l2_search.best_score_
```

```
Out[49]: 0.7276428571428571
```

```
In [50]: tfidf_search.fit(X_tr_tfidf, y_tr)
```

```
C:\Users\wurc\.conda\envs\ML\lib\site-packages\sklearn\linear_model\logistic.py:947: ConvergenceWarning: lbfgs failed to
converge. Increase the number of iterations.
  "of iterations.", ConvergenceWarning)
C:\Users\wurc\.conda\envs\ML\lib\site-packages\sklearn\linear_model\logistic.py:947: ConvergenceWarning: lbfgs failed to
converge. Increase the number of iterations.
  "of iterations.", ConvergenceWarning)
C:\Users\wurc\.conda\envs\ML\lib\site-packages\sklearn\linear_model\logistic.py:947: ConvergenceWarning: lbfgs failed to
converge. Increase the number of iterations.
  "of iterations.", ConvergenceWarning)
C:\Users\wurc\.conda\envs\ML\lib\site-packages\sklearn\linear_model\logistic.py:947: ConvergenceWarning: lbfgs failed to
converge. Increase the number of iterations.
  "of iterations.", ConvergenceWarning)
```

```
Out[50]: GridSearchCV(cv=5, error_score='raise-deprecating',
                    estimator=LogisticRegression(C=1.0, class_weight=None, dual=False,
                                                fit_intercept=True,
                                                intercept_scaling=1, l1_ratio=None,
                                                max_iter=500, multi_class='warn',
                                                n_jobs=None, penalty='l2',
                                                random_state=None, solver='lbfgs',
                                                tol=0.0001, verbose=0,
                                                warm_start=False),
                    iid='warn', n_jobs=None,
                    param_grid={'C': [1e-05, 0.001, 0.1, 1.0, 10.0, 100.0]},
                    pre_dispatch='2*n_jobs', refit=True, return_train_score=True,
                    scoring=None, verbose=0)
```

```
In [51]: tfidf_search.best_score_
```

```
Out[51]: 0.7340714285714286
```

What regularization parameters are best for each method?

```
In [52]: bow_search.best_params_
```

```
Out[52]: {'C': 0.1}
```

```
In [53]: l2_search.best_params_
```

```
Out[53]: {'C': 1.0}
```

```
In [54]: tfidf_search.best_params_
```

```
Out[54]: {'C': 0.001}
```

Let's check one of the grid search outputs to see how it went:

```
In [55]: bow_search.cv_results_
```

```
Out[55]: {'mean_fit_time': array([0.14379988, 0.21599741, 1.66459932, 4.06620111, 7.41766043,
    7.58538895]),
  'std_fit_time': array([0.01511577, 0.02066551, 0.09023791, 0.2676194 , 0.31160726,
    0.7963182 ]),
  'mean_score_time': array([0.002598 , 0.00240026, 0.00260324, 0.0028017 , 0.00260191,
    0.00320039]),
  'std_score_time': array([0.00049296, 0.00049049, 0.00049121, 0.00074931, 0.00049081,
    0.00147161]),
  'param_C': masked_array(data=[1e-05, 0.001, 0.1, 1.0, 10.0, 100.0],
    mask=[False, False, False, False, False, False],
    fill_value='?',
    dtype=object),
  'params': [{'C': 1e-05},
    {'C': 0.001},
    {'C': 0.1},
    {'C': 1.0},
    {'C': 10.0},
    {'C': 100.0}],
  'split0_test_score': array([0.56535714, 0.70464286, 0.71285714, 0.69785714, 0.67857143,
    0.66785714]),
  'split1_test_score': array([0.55 , 0.71178571, 0.71964286, 0.7075 , 0.68178571,
    0.66821429]),
  'split2_test_score': array([0.57285714, 0.70892857, 0.71642857, 0.70642857, 0.68321429,
    0.66892857]),
  'split3_test_score': array([0.56857143, 0.72035714, 0.725 , 0.69821429, 0.67964286,
    0.67035714]),
  'split4_test_score': array([0.54535714, 0.71392857, 0.72107143, 0.70428571, 0.67857143,
    0.66321429]),
  'mean_test_score': array([0.56042857, 0.71192857, 0.719 , 0.70285714, 0.68035714,
    0.66771429]),
  'std_test_score': array([0.01077933, 0.00523723, 0.00412434, 0.00407206, 0.00184888,
    0.00240747]),
  'rank_test_score': array([6, 2, 1, 3, 4, 5]),
  'split0_train_score': array([0.56491071, 0.73678571, 0.86758929, 0.95883929, 0.99491071,
    1. ]),
  'split1_train_score': array([0.55928571, 0.73589286, 0.86803571, 0.96035714, 0.995 ,
    1. ]),
  'split2_train_score': array([0.56482143, 0.73785714, 0.86598214, 0.95964286, 0.99553571,
    1. ]),
  'split3_train_score': array([0.56848214, 0.73357143, 0.86669643, 0.95982143, 0.995625 ,
    1. ]),
  'split4_train_score': array([0.56482143, 0.73428571, 0.86625 , 0.95785714, 0.99464286,
    1. ]),
  'mean_train_score': array([0.56446429, 0.73567857, 0.86691071, 0.95930357, 0.99514286,
    1. ]),
  'std_train_score': array([0.0029467 , 0.00157467, 0.00078368, 0.0008719 , 0.00037712,
    0. ])}
```

```
In [56]: import pickle
```

```
In [57]: results_file = open('tfidf_gridcv_results.pkl', 'wb')
pickle.dump(bow_search, results_file, -1)
pickle.dump(tfidf_search, results_file, -1)
pickle.dump(l2_search, results_file, -1)
results_file.close()
```

```
In [58]: pkl_file = open('tfidf_gridcv_results.pkl', 'rb')
bow_search = pickle.load(pkl_file)
tfidf_search = pickle.load(pkl_file)
l2_search = pickle.load(pkl_file)
pkl_file.close()
```

```
In [59]: search_results = pd.DataFrame.from_dict({'bow': bow_search.cv_results_['mean_test_score'],
  'tfidf': tfidf_search.cv_results_['mean_test_score'],
  'l2': l2_search.cv_results_['mean_test_score']})
search_results
```

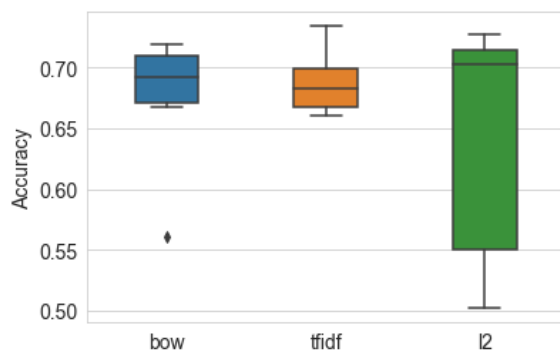
```
Out[59]:
```

	bow	tfidf	l2
0	0.560429	0.701643	0.502500
1	0.711929	0.734071	0.502500
2	0.719000	0.692357	0.710357
3	0.702857	0.673286	0.727643
4	0.680357	0.665714	0.716286
5	0.667714	0.660714	0.694786

Plot cross validation results

```
In [60]: %matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("whitegrid")
```

```
In [61]: ax = sns.boxplot(data=search_results, width=0.4)
ax.set_ylabel('Accuracy', size=14)
ax.tick_params(labelsize=14)
plt.savefig('tfidf_gridcv_results.png')
```



```
In [62]: m1 = simple_logistic_classify(X_tr_bow, y_tr, X_te_bow, y_te, 'bow',
    _C=bow_search.best_params_['C'])
m2 = simple_logistic_classify(X_tr_l2, y_tr, X_te_l2, y_te, 'l2-normalized',
    _C=l2_search.best_params_['C'])
m3 = simple_logistic_classify(X_tr_tfidf, y_tr, X_te_tfidf, y_te, 'tf-idf',
    _C=tfidf_search.best_params_['C'])
```

Test score with bow features: 0.7293333333333333

Test score with l2-normalized features: 0.739

Test score with tf-idf features: 0.7413333333333333

```
In [63]: bow_search.cv_results_['mean_test_score']
```

```
Out[63]: array([0.56042857, 0.71192857, 0.719      , 0.70285714, 0.68035714,
0.66771429])
```

```
In [ ]:
```

```
In [ ]:
```